Fully Loaded and Incremental Loaded Azure Data Pipeline Project

Overview:

This project outlines the development and deployment of a robust data pipeline on Microsoft Azure. The pipeline is designed to efficiently ingest, transform, and load data from source systems into an analytical data store, utilising both **Fully Loaded** and **Incremental Loaded** methodologies to ensure data integrity, performance, and cost-effectiveness.

Goal:

To establish a scalable and reliable data integration solution in Azure that supports business intelligence and reporting requirements by providing up-to-date, harmonised data.

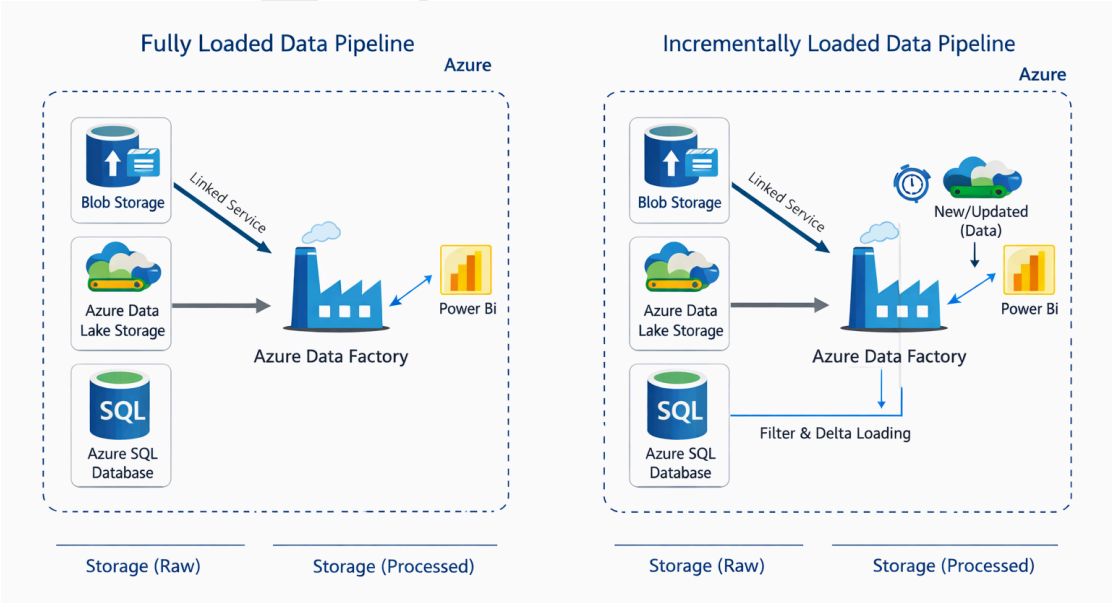
Core Subject:

Azure Data Factory (ADF), Azure Synapse Analytics (or Azure SQL Database/Azure Cosmos DB), Azure Blob Storage, and associated data loading strategies (Full vs. Incremental).

Target Audience:

Data Engineers, Data Architects, Project Managers, and Stakeholders are interested in the technical implementation and operational status of the data pipeline.

Project Architecture:



Sinetti Balu

Data Loading Strategies:

Effective data loading requires distinct approaches depending on the data source characteristics, volume, and business requirements.

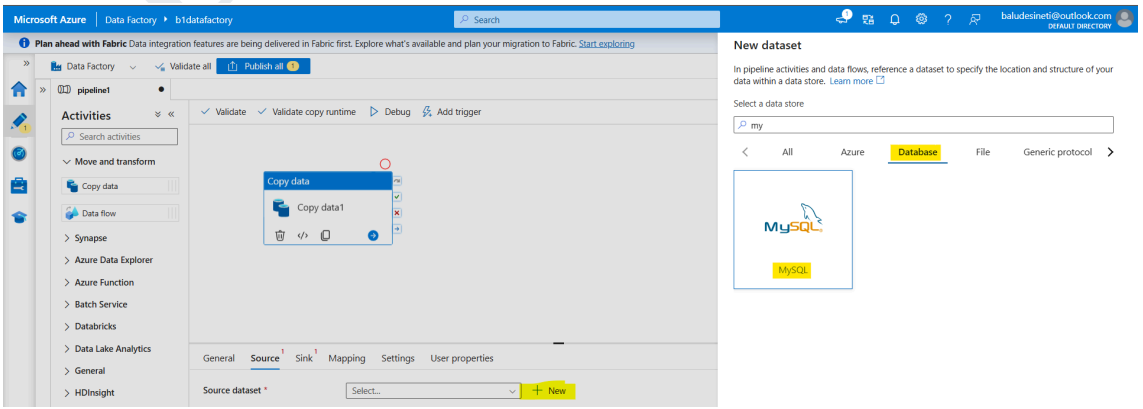
Fully Loaded (Initial Load):

The fully loaded approach involves extracting all data from the source system and loading it into the destination. This is typically performed once at the start of the project or when a complete data refresh is required.

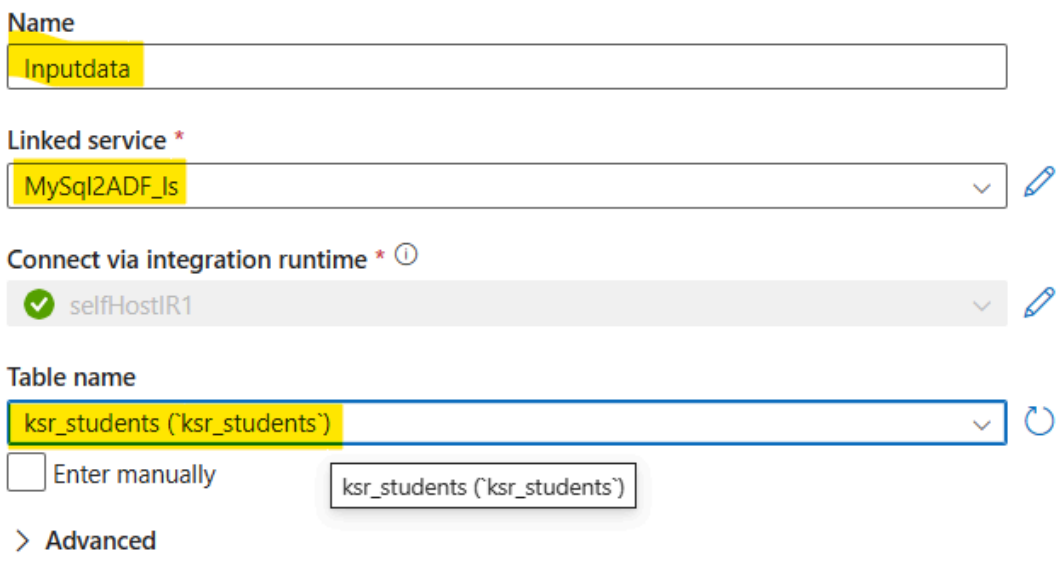
Aspect	Description	Use Case
Definition	Extracts and loads *all* records from the source table/system.	Initial population of data warehouse, small dimension tables, or periodic full refresh of small datasets.
Mechanism	Simple copy activity in ADF, bulk insert via Synapse PolyBase or dedicated SQL pool.	Guaranteed data synchronisation but high resource usage.
Frequency	Once (initial setup) or infrequently (full refresh).	Low frequency.
Performance	High latency, high initial resource cost.	Can be optimised with parallel loading techniques.

Creating the new Pipeline:

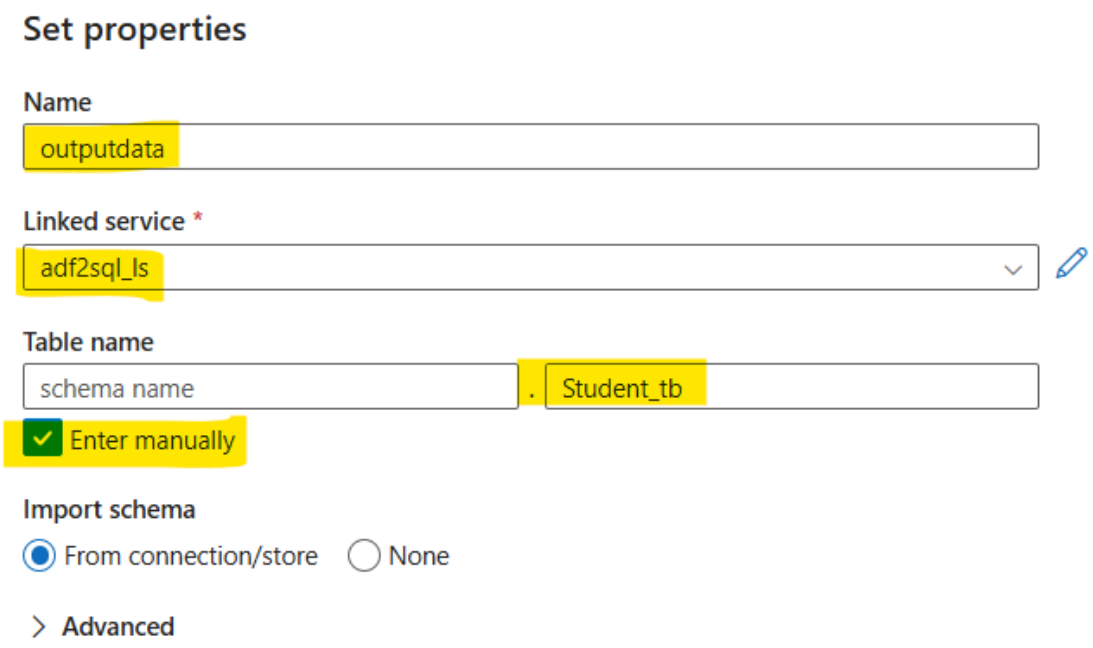
- In the copy activity's source tab, select a new dataset and establish a connection to MySQL.
- This action opens the set properties.
- Name the input dataset "input data."
- Utilise "mysql2adf_ls" as the linked service.
- Specify the "ksr_students" table for the operation.



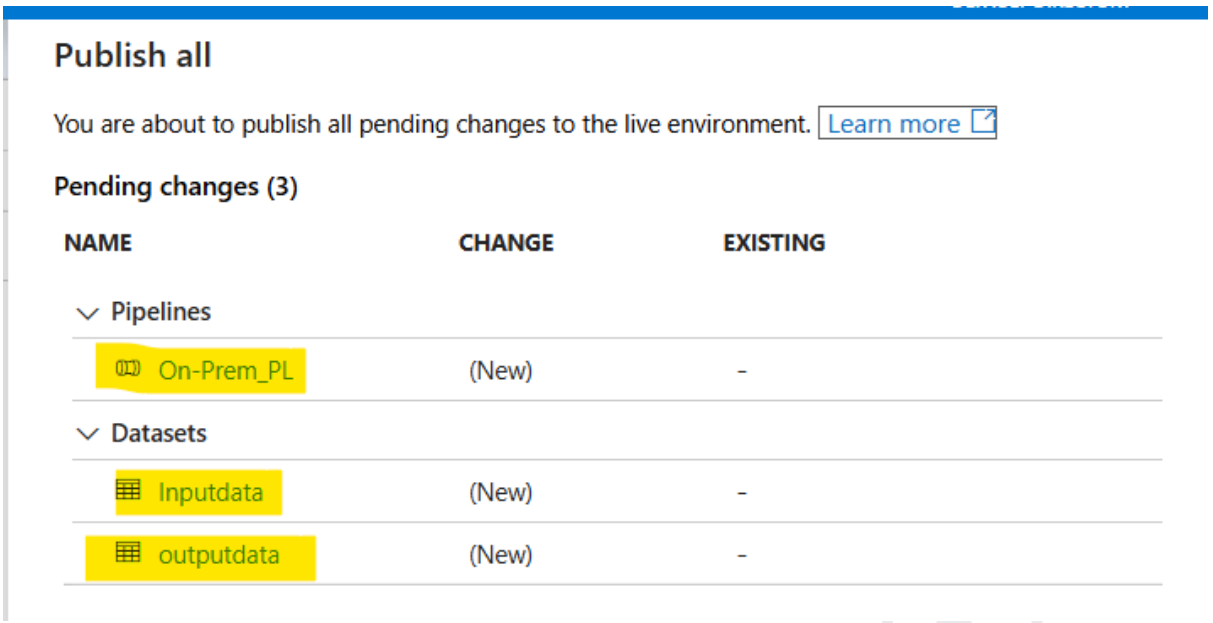
Designed By Bali

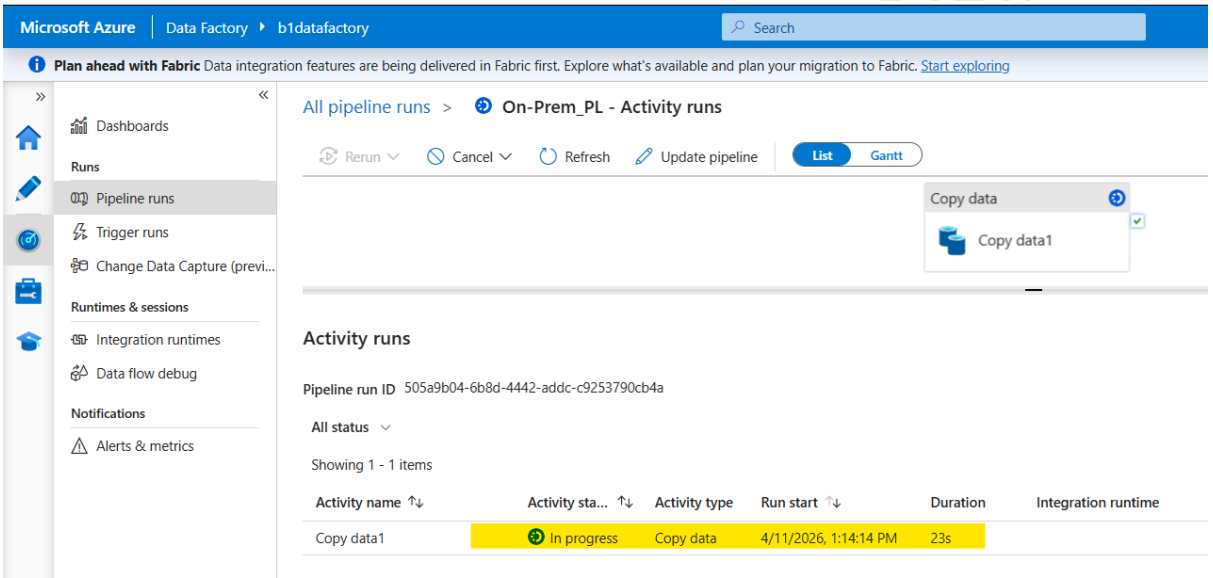


- In the copy activity's sink tab, select a new dataset and establish a connection to Azure SQL Database.
- This action opens the set properties.
- Name the output dataset "output data."
- Utilise "adf2sqk_ls" as the linked service.
- Specify the "Student_tb" table for the operation.



- Upon the completion of all steps, publish the work.
- Execute the pipelines.





- The pipeline was executed successfully.
- It processed 7,000 records into the student table.
- The next step is to verify the presence of these records within the MySQL database.

Details Refresh

Learn more on copy performance details from here.

Activity run id: bf457450-b8fc-48ee-8491-6f93049163de

MySQL → Succeeded → Azure SQL Database

Self-hosted IR: selfHostIR1

Region: West US 2

Data read:	1.963 MB	Data written:	1.963 MB
Rows read:	7,000	Rows written:	7,000
Peak connections:	1	Peak connections:	3

Copy duration	00:00:45
Throughput:	57.727 KB/s

MySQL → Azure SQL Database

Start time	4/11/2026, 1:14:15 PM
Used parallel copies	1
Duration	00:00:45

Details		Working duration	Total duration
Queue			00:00:03
Pre copy script			00:00:02
Transfer	Time to first byte	00:00:00	00:00:32
	Reading from source	00:00:00	
	Writing to sink	00:00:19	

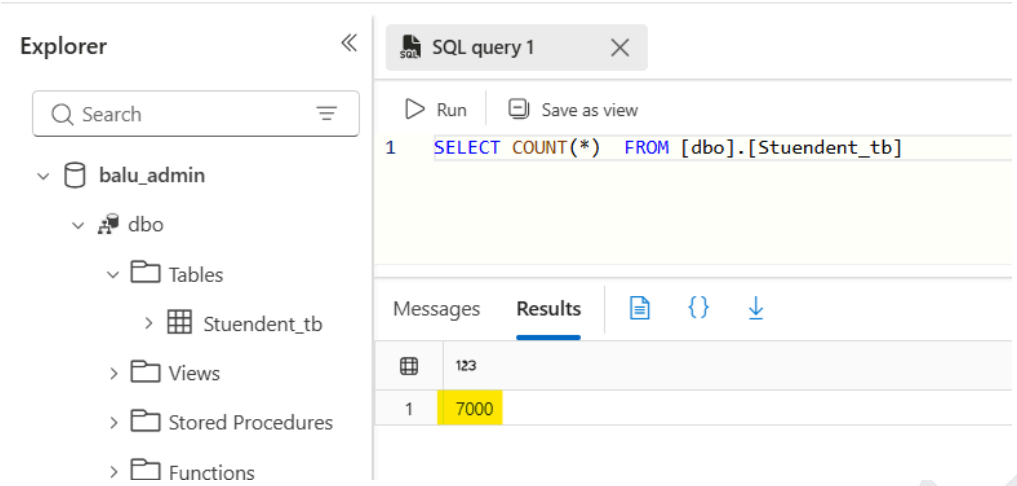
Data consistency verification Not verified

```
100 • select count(*) from ksr_students;
101 • create table student inc th as(sel
```

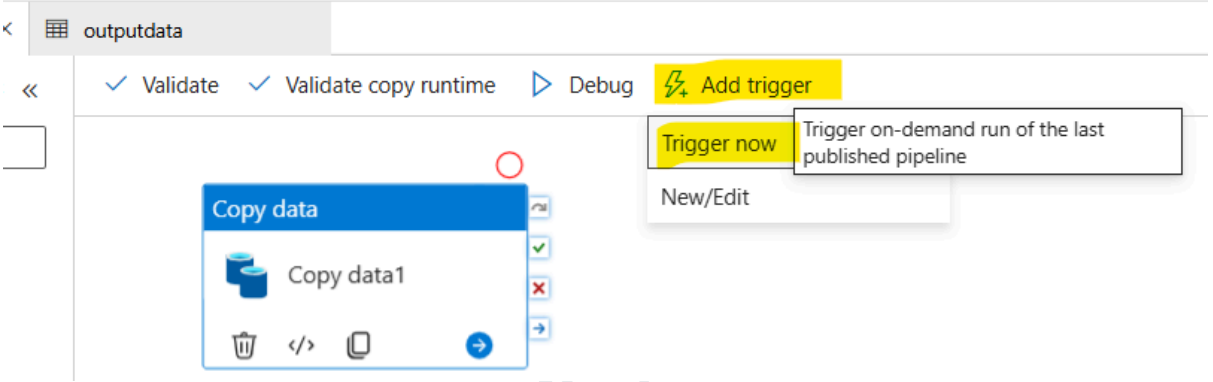
Result Grid | Filter Rows: | Ex

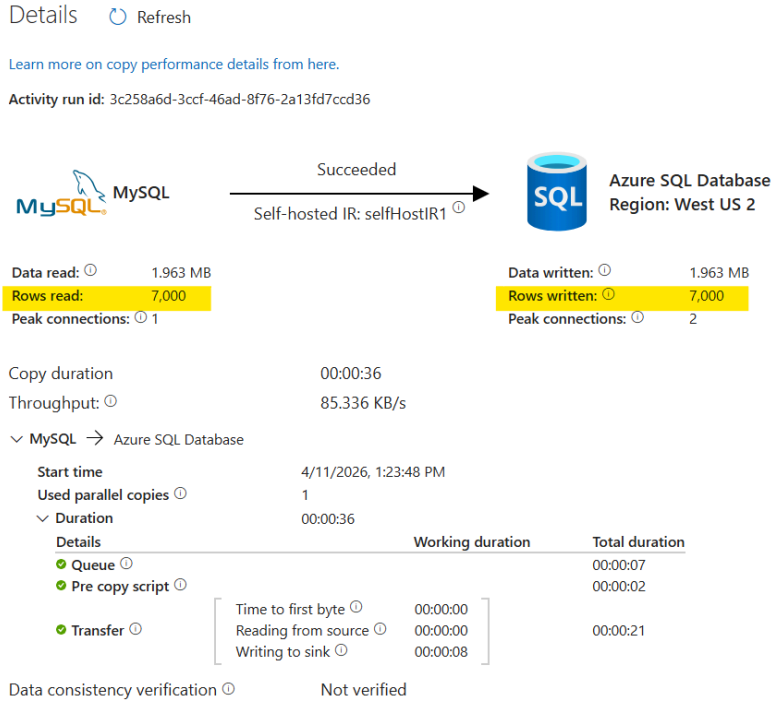
count(*)
7000

The Azure SQL database additionally reports 7,000 records.

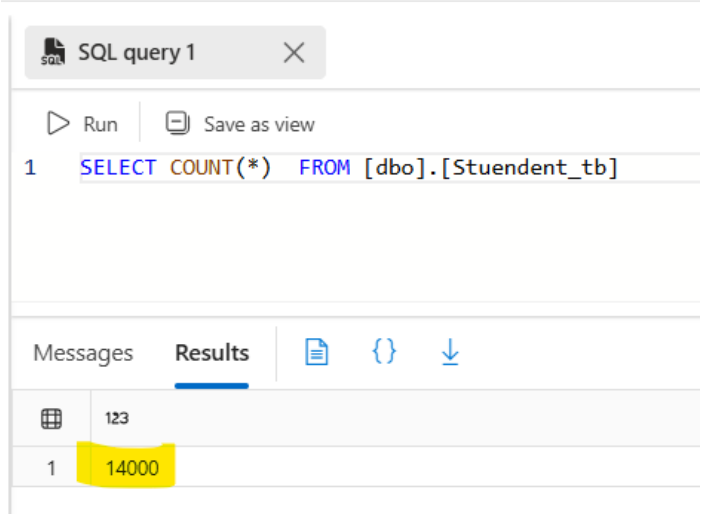


Rerun the pipeline.





The pipeline has been successfully re-executed. We can now check the results within the Azure SQL database.



- The data pipeline is facing two critical issues:
  - **Excessive volume:** 14,000 records, which is double the required 7,000.
  - **Duplication.**
- These issues severely compromise data integrity and operational efficiency, resulting in:
  - Slower processing.
  - Higher resource utilisation.
  - Inaccurate Business Intelligence (BI) reports.

Data Pipeline Daily Record Summary

Day	New Records Processed	Cumulative Total Records
Day 1	7000	7000
Day 2	500	7500
Day 3	1000	8500

Incremental Loaded (Ongoing Updates):

The incremental loaded approach only extracts and loads data that has changed or been newly created since the last successful pipeline run. This is the preferred method for ongoing data synchronisation due to its efficiency.

- A new MySQL table, `student_inc_tb`, is being created.
- It is derived from the `kst_student` table.
- The new table will include an extra column: `last_updated_date`.
- The initial value for `last_updated_date` is set to ' 2026-04-10 '.

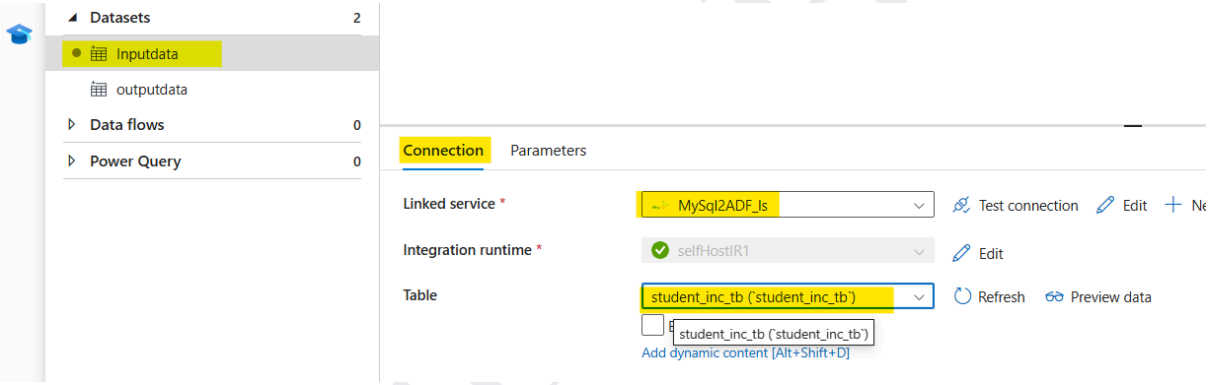
Desmeti Balu

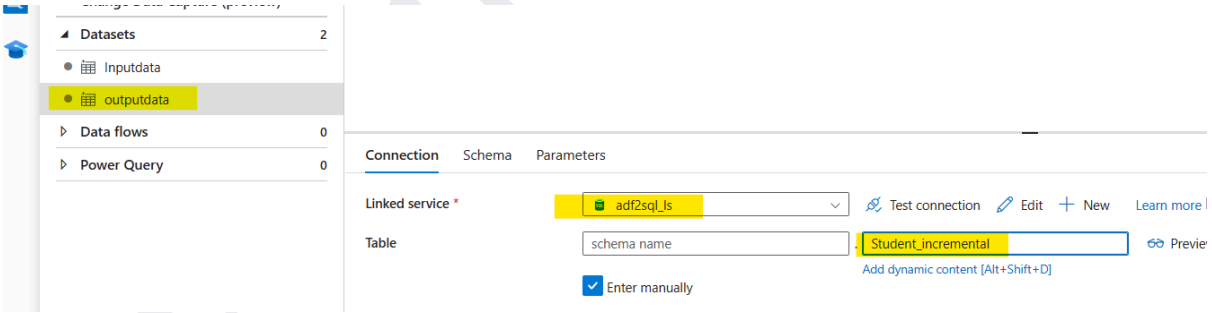
```
create table student_inc_tb as(select * from ksr_students);
select count(*) from student_inc_tb; #7000
alter table student_inc_tb add column last_updated_date date;
set sql_safe_updates = 0;
update student_inc_tb set last_updated_date = '2026-04-10';
```

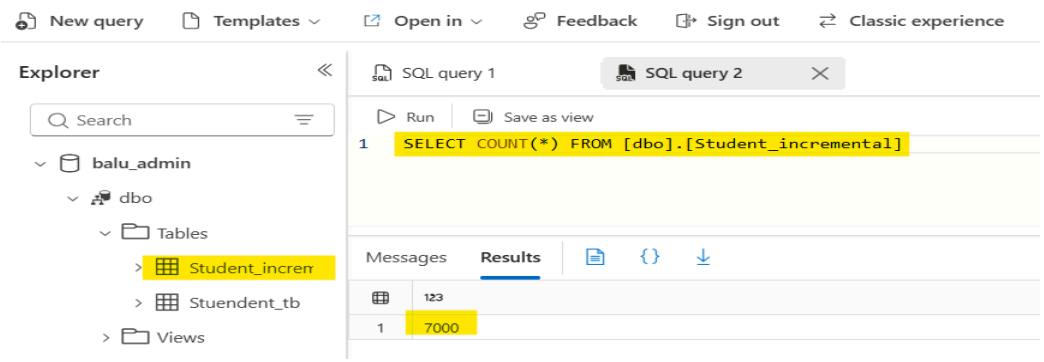
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

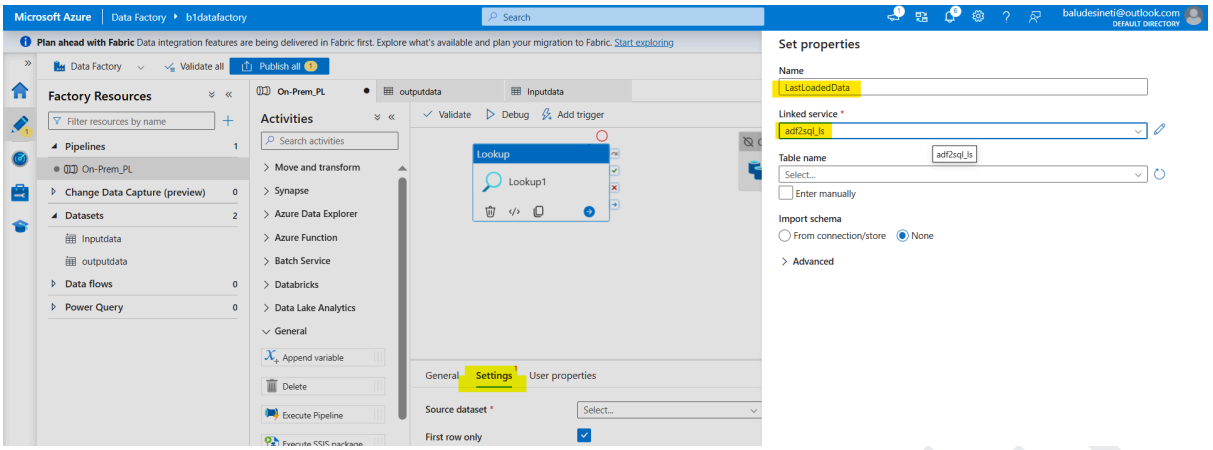
count(*)
7000

- Update the input data by changing the table name to the newly created `student_inc_tb`.
- Modify the output data to `Student_incremental`.
- Execute the pipeline.

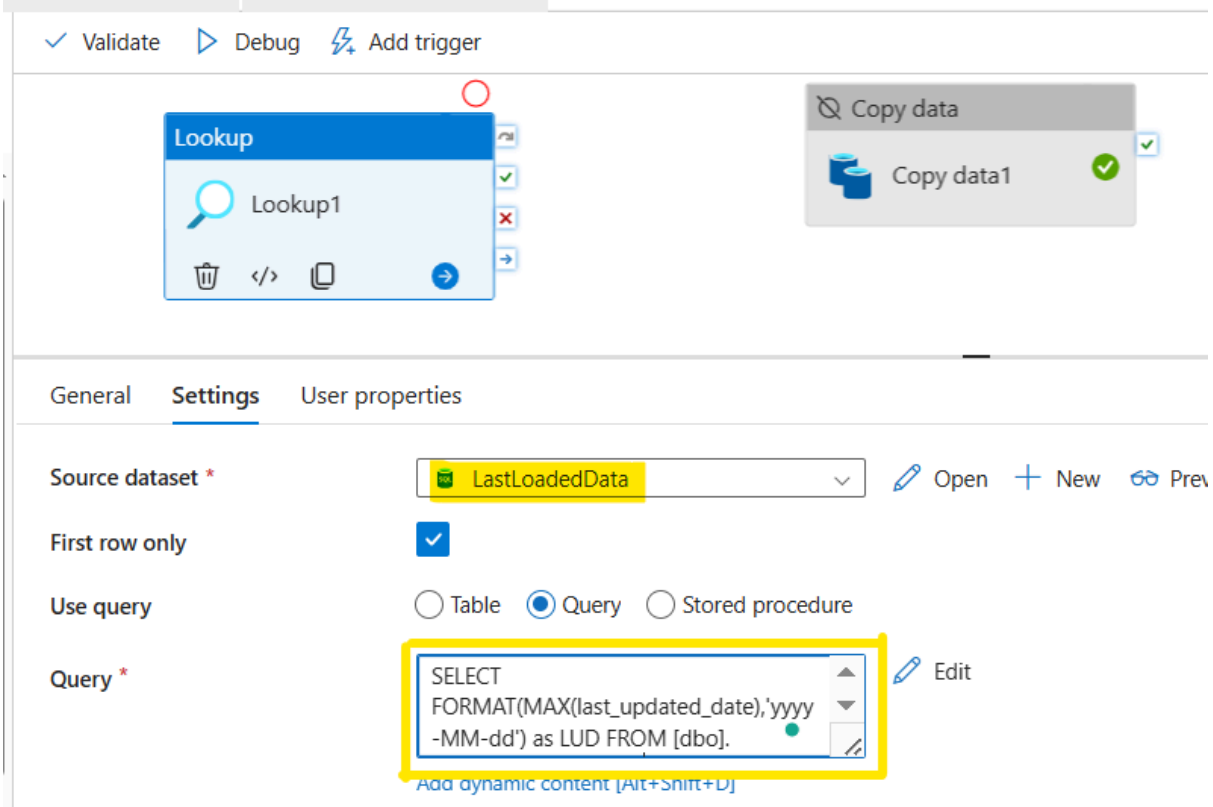


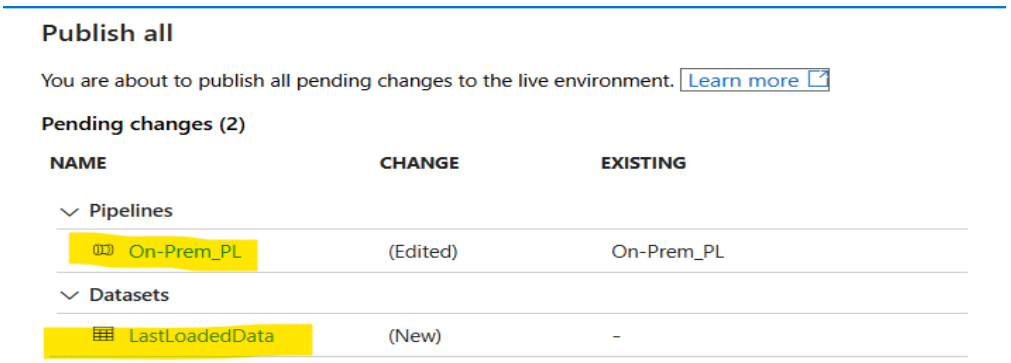


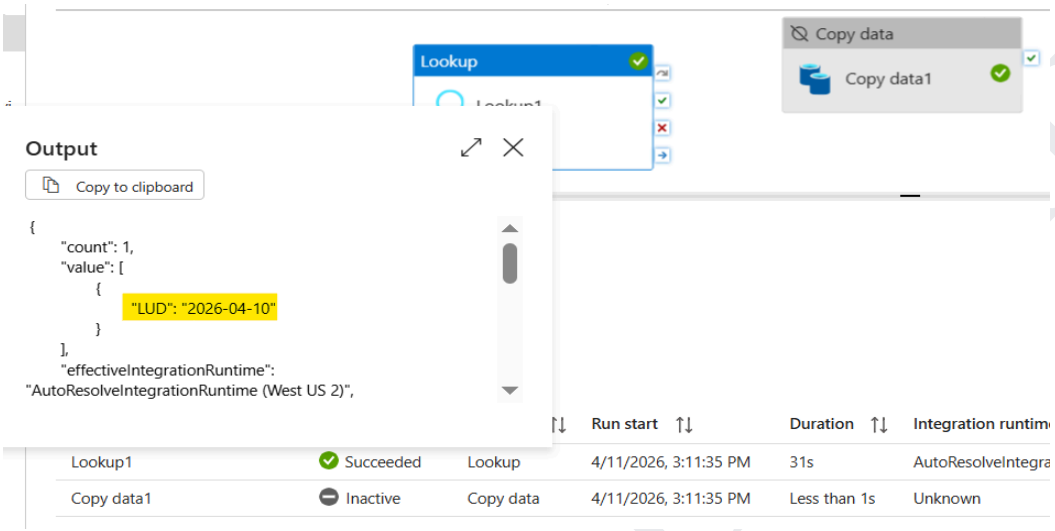




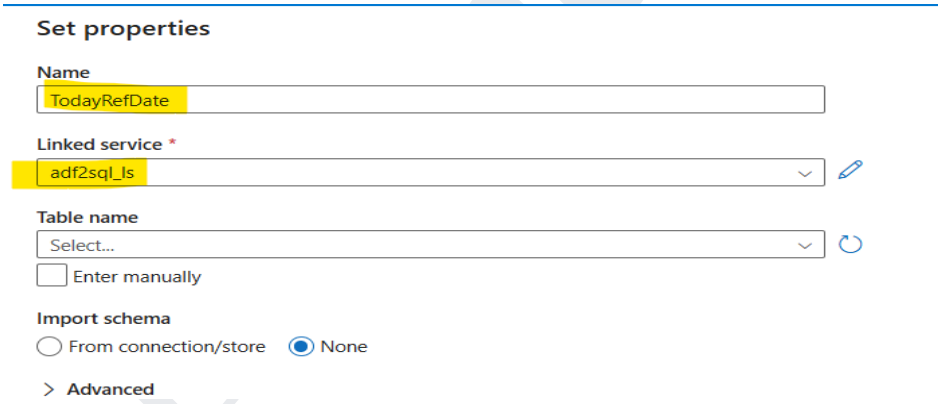
- **Lookup Query:** Retrieves the last update date.
- **SQL:**
  SELECT FORMAT(MAX(last_updated_date), 'yyyy-MM-dd') as LUD
- FROM [dbo].[Student_incremental]
- **Function:** Finds the most recent `last_updated_date` from the `[dbo].[Student_incremental]` table.
- **Format:** Formats the date as 'yyyy-MM-dd'.
- **Alias:** The result is aliased as `LUD`.



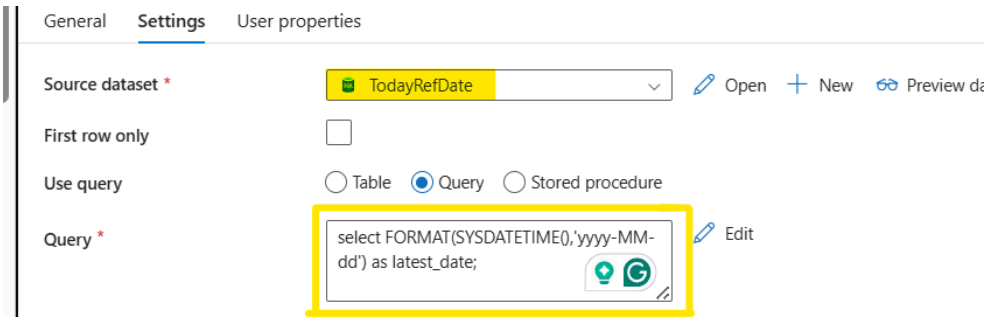


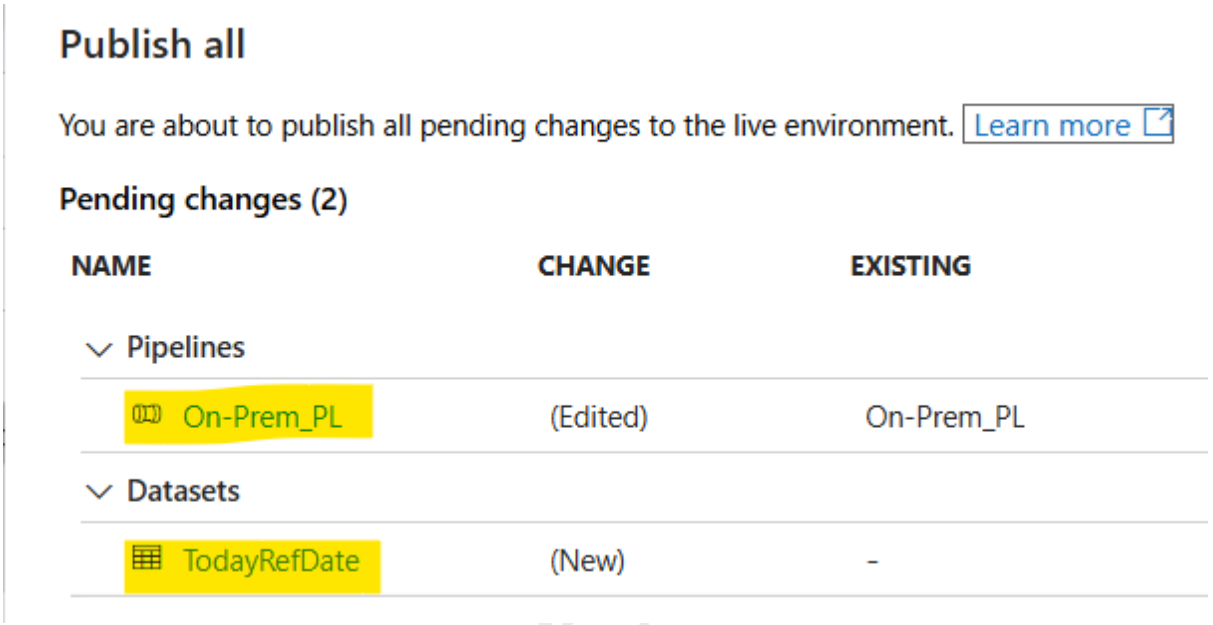


Integrate an additional lookup operation to retrieve the most current information for today.

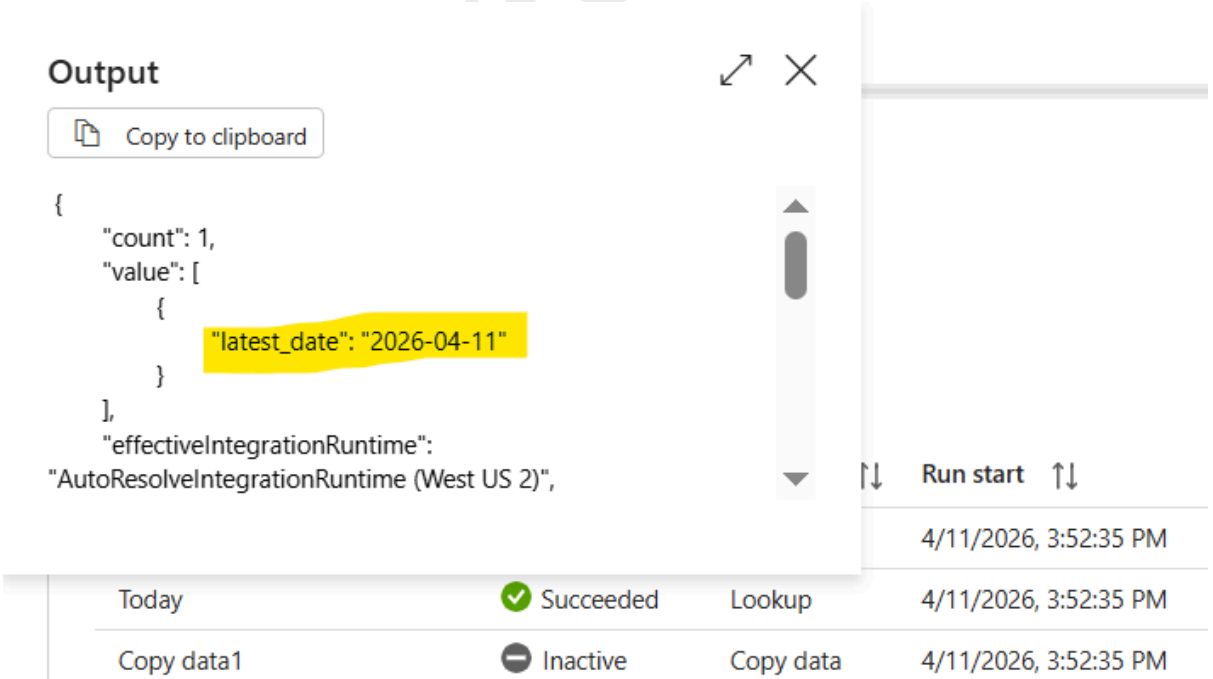


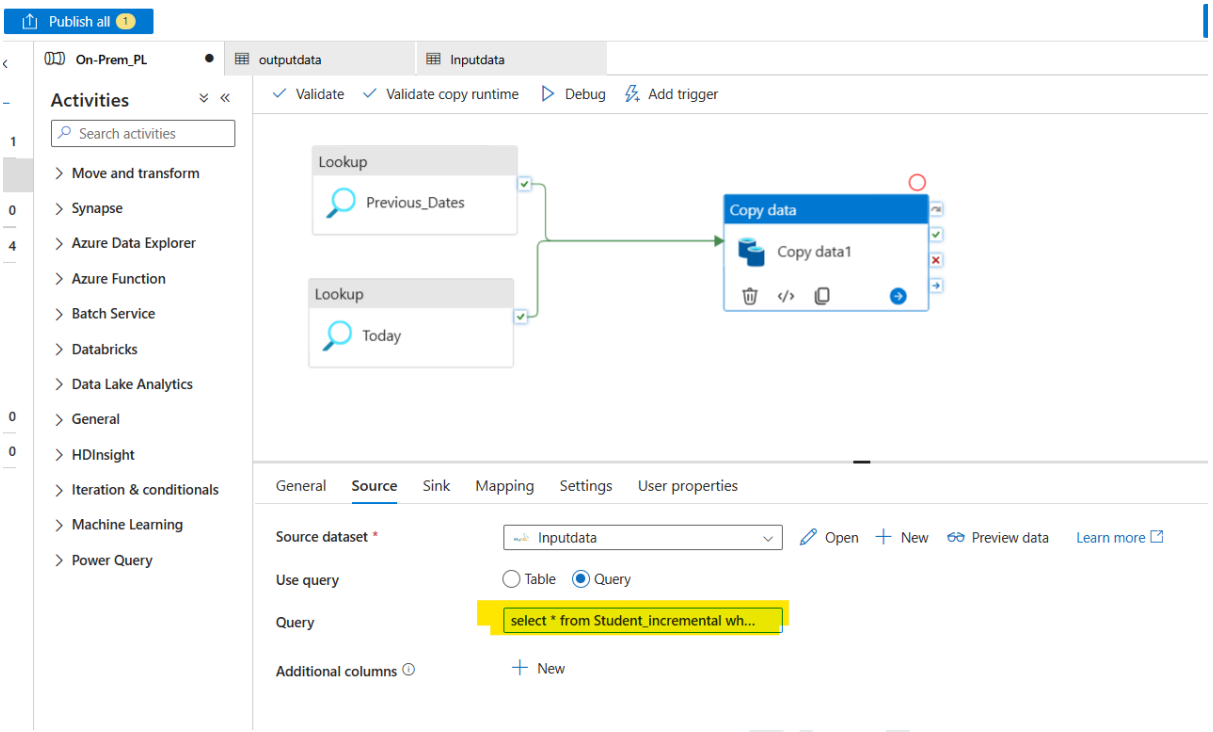
Construct a query in the second lookup activity to retrieve the required updates. For instance, you can get the current date using the following SQL:
select FORMAT(SYSDATETIME(),'yyyy-MM-dd') as latest_date;





Once all updates are complete, the pipeline should be published.





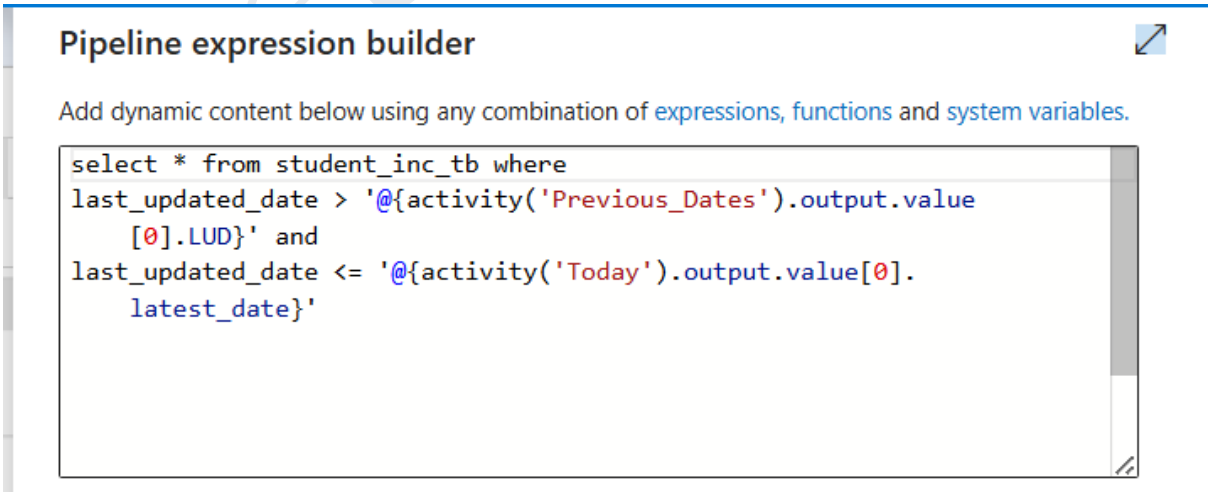
- Once the two lookup activities have successfully executed, activate the 'Copy Data' activity.
- This activity should join the results from both lookups to construct the following dynamic query:

SELECT * FROM student_inc_tb

WHERE last_updated_date > '@{activity('Previous_Dates').output.value[0].LUD}'

AND last_updated_date <= '@{activity('Today').output.value[0].latest_date}'

- After writing this query, publish the data and then execute the pipeline.



Details Refresh

Learn more on copy performance details from here.

Activity run id: eafa9509-9bfa-4dac-94f0-b5ce5b3cf626

MySQL → Succeeded → Azure SQL Database

Self-hosted IR: selfHostIR1

Region: West US 2

Data read:	16.368 KB	Data written:	16.368 KB
Rows read:	62	Rows written:	62
Peak connections:	1	Peak connections:	2

Copy duration	00:00:20
Throughput:	1.364 KB/s

MySQL → Azure SQL Database

Start time	4/11/2026, 4:37:20 PM
Used parallel copies	1
Duration	00:00:20

Details		Working duration	Total duration
Queue			00:00:04
Transfer	Time to first byte	00:00:00	00:00:12
	Reading from source	00:00:00	
	Writing to sink	00:00:02	

Data consistency verification Not verified

- The data has been successfully imported.
- Verification of the records in both the MySQL and Azure SQL databases shall now be conducted.

```
112 • select last_updated_date ,count(*) from student_inc_tb
113   group by last_updated_date;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

last_updated_date	count(*)
2026-04-10	7000
2026-04-09	673
2026-04-11	62
2026-04-12	673

```sql
SELECT FORMAT((last_updated_date),'yyyy-MM-dd'),count(last_updated_date) FROM [dbo].[Student_incremental]
group by FORMAT((last_updated_date),'yyyy-MM-dd')
```

Messages | Results

	ABC	123
1	2026-04-11	62
2	2026-04-10	7000

High-Level Workflow:

1. **Ingestion (ADF):**
   - **Full Load:** Copy all source data into the Landing Zone (Blob Storage).
   - **Incremental Load:** Query source based on the stored watermark value; copy only delta records to the Landing Zone.
2. **Transformation (ADF Mapping Data Flows/Synapse):**
   - Data is cleansed, standardised, and transformed (e.g., joins, aggregations) in the Staging area (Synapse/Blob).
3. **Loading (Synapse):**
   - **Full Load:** Truncate the target table and reload all data.
   - **Incremental Load:** Load delta records using `INSERT` or `MERGE` statements into the final analytical tables. Update the control table with the new high-watermark value.

Project Phases and Roadmap:

Phase	Description	Key Deliverables
Phase 1: Setup & Foundation	Provision core Azure resources, establish networking, and configure security (Key Vault).	Azure environment provisioned, ADF linked services configured, Security baseline documented.
Phase 2: Full Load Implementation	Develop and test the Full Load pipeline for all critical source tables.	Full Load ADF pipelines for critical tables are functional. Initial data load completed.
Phase 3: Incremental Load Implementation	Implement CDC mechanisms (Watermark, Change Tracking) and develop incremental pipelines.	Incremental Load ADF pipelines deployed, Watermark control table in use, Delta processing logic validated.
Phase 4: Optimisation & Go-Live	Performance tuning, robust error handling, monitoring setup, and final UAT.	Performance metrics achieved, Comprehensive monitoring dashboards, and production deployment.

Desineti Balu